

I2236INTRODUCTION TO EMBEDDED SYSTEMS

LECTURE 1 INTRODUCTION & FOUNDATIONS

Bilal Daass
04/02/2026



COURSE OVERVIEW

Module 1: Introduction & Foundations (Week 1–2)

Module 2: Arduino Ecosystem & Programming Basics (Week 3–4)

Module 3: Digital & Analog Interfacing (Week 5–6)

Module 4: Interfacing Peripherals (Week 7–8)

Module 5: Sensors & Actuators (Week 9–10)

Module 6: Communication & Integration (Week 11–12)

Module 7: Project (Week 13–15)

MODULE 1 : INTRODUCTION & FOUNDATION

- **Useful Info**
- **Embedded Systems: categories, attributes, constraints**
- **Design process & product life cycle**
- **Overview of microcontrollers: processor, memories, and interfaces**
- **AVR Architecture (ATmega328, ATmega2560)**

USEFUL INFO

- **Course requirements**

- A laptop with an internet connection.
- An Arduino Kit.
- A logbook covering the students' work during the semester.

- **Assessment**

Evaluation	Weight
Lab Assignments / Mini-projects	20%
Participation / In-class activities	10%
Final Exam	40%
Final Project (Prototype + Report + Presentation)	30%

USEFUL INFO

- **Project and Presentation Guidelines**

- Teams must consist of 2 or 3 students.
- Teams should meet regularly to develop their presentations.
- Teams are required to submit a written report, a PowerPoint presentation, and a demo video.

- **Deadlines**

Course Activity	Week
Team composition	Week 3
Project Idea	Week 5
Project submission	Last week

USEFUL INFO

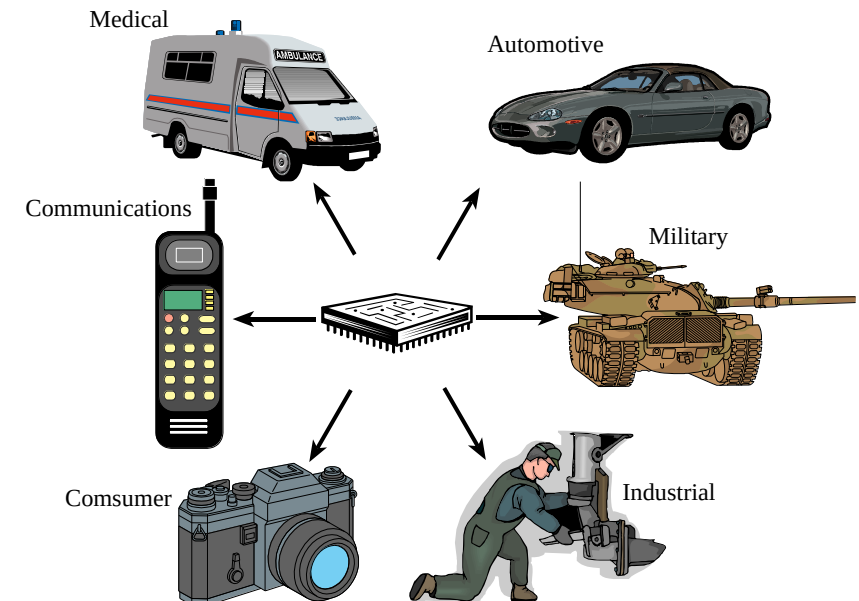
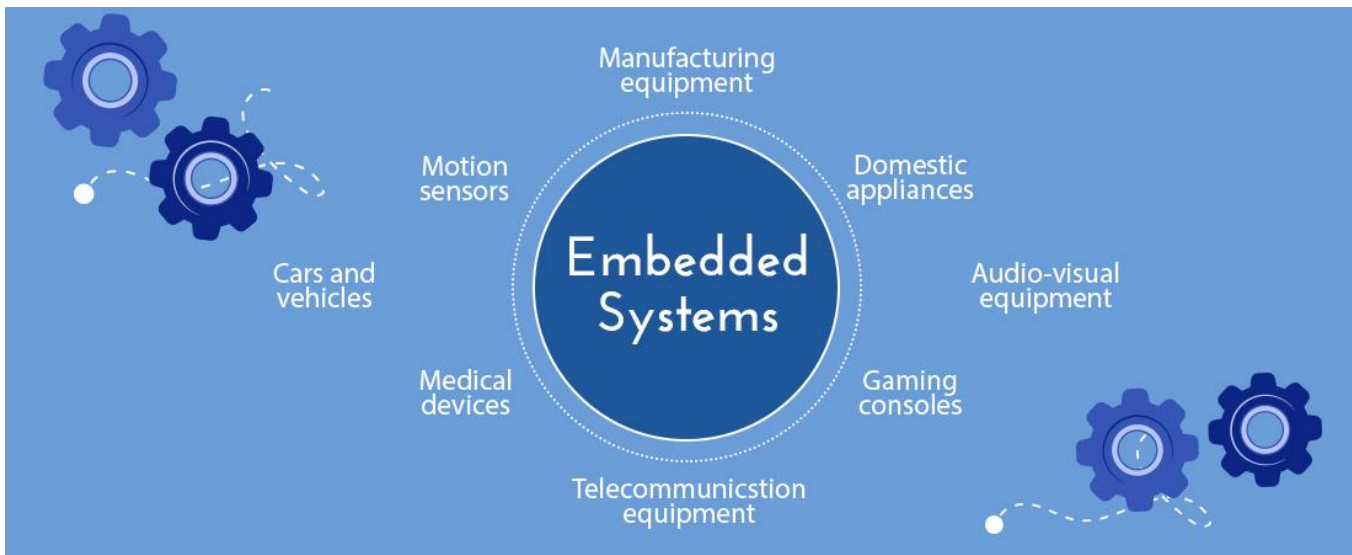
- **Combines concepts, skills, and experience**
 - Build real systems, bridge physical and virtual. Embedded systems critical to everyone, knowledge gained goes way beyond that.
 - Learn to how a system works and development flow, learn how a computer and embedded system works practically, learn how to program, learn how to use programming tools and mixed environments, learn about interfacing and design
- **Bibliography**
 - Arduino Applied, Neil Cameron, 1st Edition, Apress, 2019. ISBN: 978-1-4842-3959-9
 - Designing Embedded Systems with Arduino, Tianhong Pan and Yi Zhu, 1st Edition, Springer Nature, 2018. ISBN 978-981-10-4417-5

No labs this week!

EMBEDDED SYSTEMS: CATEGORIES, ATTRIBUTES, CONSTRAINTS

What is Embedded System?

An Electronic/Electro mechanical system which is designed to perform a specific function and is a combination of both hardware and firmware (Software) E.g. Electronic Toys, Mobile Handsets, Washing Machines, Air Conditioners, Automotive Control Units, Set Top Box, DVD Player etc



EMBEDDED SYSTEMS: CATEGORIES, ATTRIBUTES, CONSTRAINTS

Attributes

- **Reliability:** is the probability that a system will perform its function without failure over a specific period.
- **Maintainability :** refers to the ease with which a system can be repaired or updated (e.g. Modular code for easy debugging.).
- **Availability :** is the percentage of time a system is operational and accessible when needed/
- **Safety:** Safety ensures that a system failure does not result in catastrophic physical damage or injury to humans.
- **Power Efficiency :** Especially for battery-operated devices, the ability to minimize power consumption.

EMBEDDED SYSTEMS: CATEGORIES, ATTRIBUTES, CONSTRAINTS

Constraints



- **Cost:** In mass production (e.g., making 1 million smart meters), saving \$0.10 on a microcontroller saves the company \$100,000. We are always fighting to use the cheapest hardware that gets the job done.



- **Power Consumption:** For battery-operated devices, power is the most expensive resource. We optimize code to minimize CPU cycles.

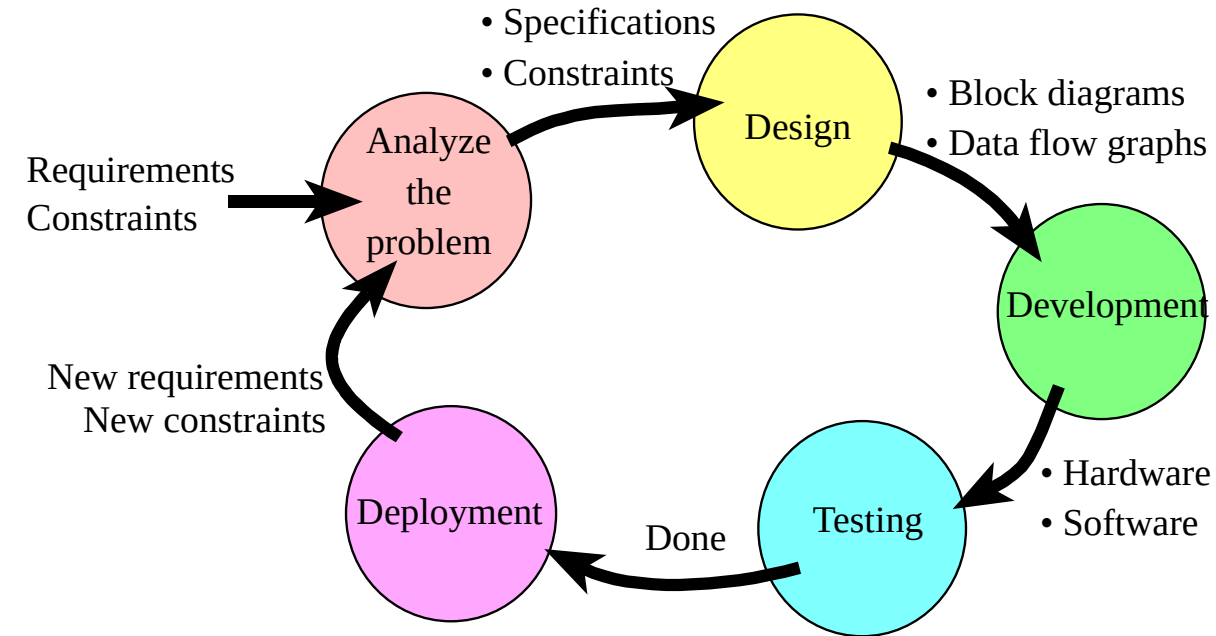


- **Memory/Size:** We operate in "kilobytes" or "megabytes," not "terabytes." Efficient data structures and memory management are non-negotiable.



- **Safety:** If your PC crashes, you reboot it. If a car's ABS (Anti-lock Braking System) crashes, lives are at stake. These systems must be robust and often include "Watchdog Timers" to reset themselves if the code hangs.

DESIGN PROCESS & PRODUCT LIFE CYCLE



Analysis (What?)

- Requirements -> Specifications

Design (How?)

- High-Level: Block Diagrams
- Engineering: Algorithms, Data Structures, Interfacing

Implementation(Real)

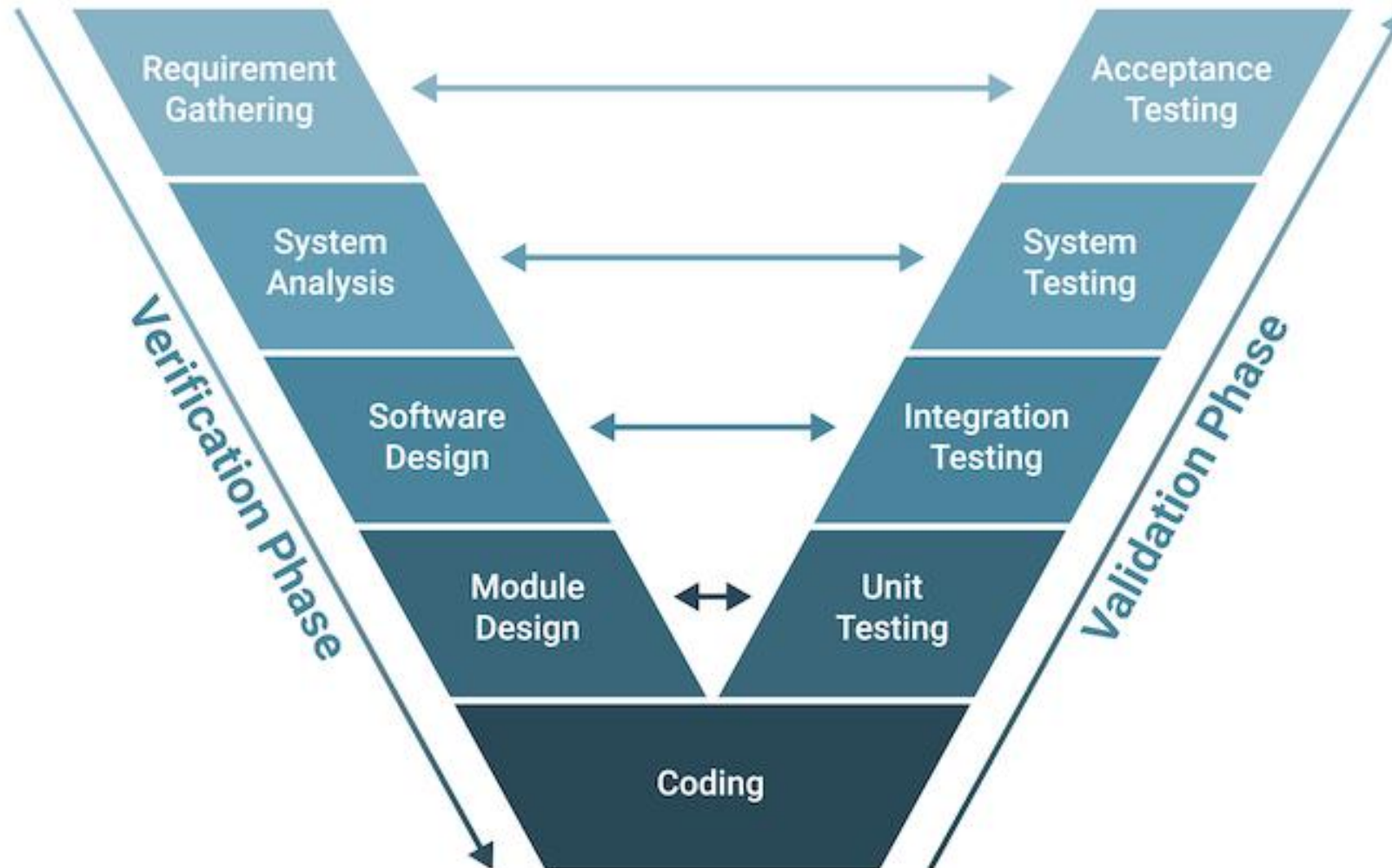
- Hardware, Software

Testing (Works?)

- Validation : Correctness
- Performance: Efficiency

Maintenance (Improve)

DESIGN PROCESS & PRODUCT LIFE CYCLE



OVERVIEW OF MICROCONTROLLERS: PROCESSOR, MEMORIES, AND INTERFACES

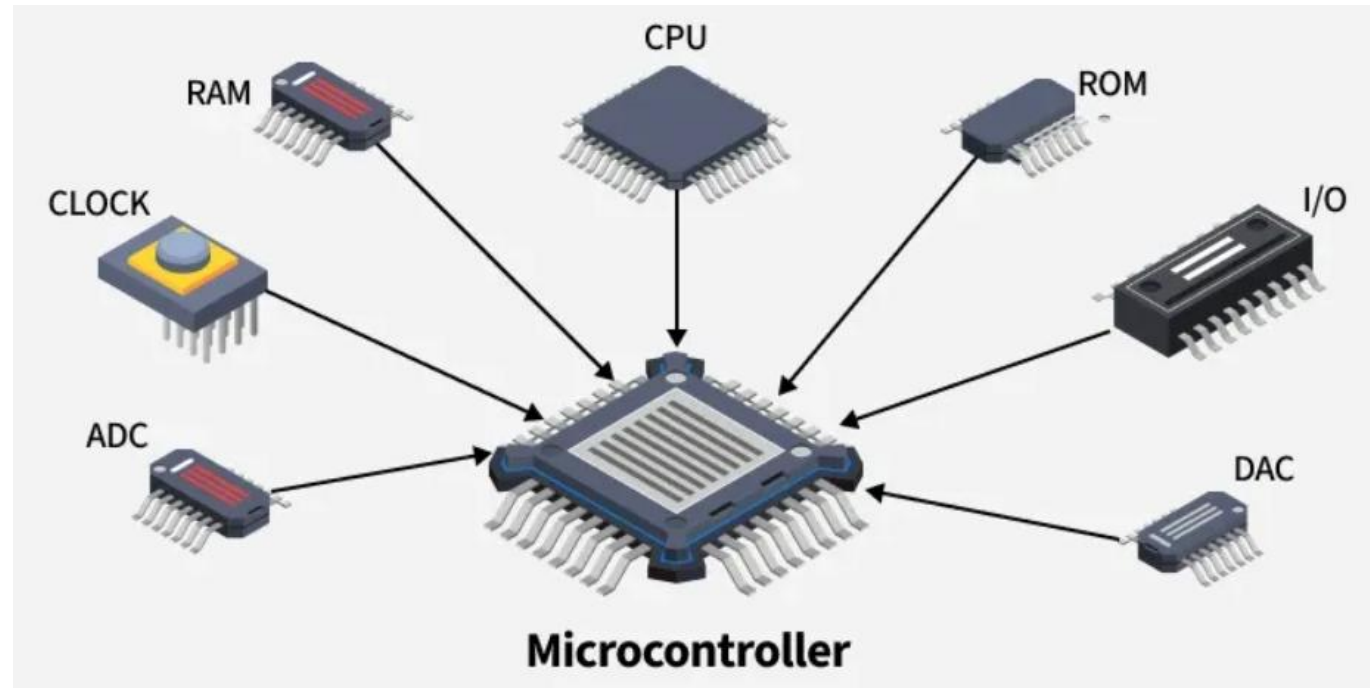
Microcontroller

A microcontroller (MCU) is a small computer on a single integrated circuit that is designed to control specific tasks within electronic systems.

It combines the functions of a central processing unit (CPU), memory, and input/output interfaces, all on a single chip.

A microcontroller includes:

- A processor that runs instructions.
- Memory to store programs and data.
- Input/Output ports to connect with buttons, sensors, screens, etc.
- Communication ports (to connect with other devices).



OVERVIEW OF MICROCONTROLLERS: PROCESSOR, MEMORIES, AND INTERFACES

Why Are Microcontrollers Useful?

- They are **programmable**, so you can make them do different tasks.
- **Common programming languages** used are C, C++, and assembly language.
- They are often **low-power**, which is great for devices that run on batteries.
- They are **small, cheap**, and perfect for **controlling devices** over a long time.
- Most microcontrollers are **embedded**, meaning they are built inside other devices.

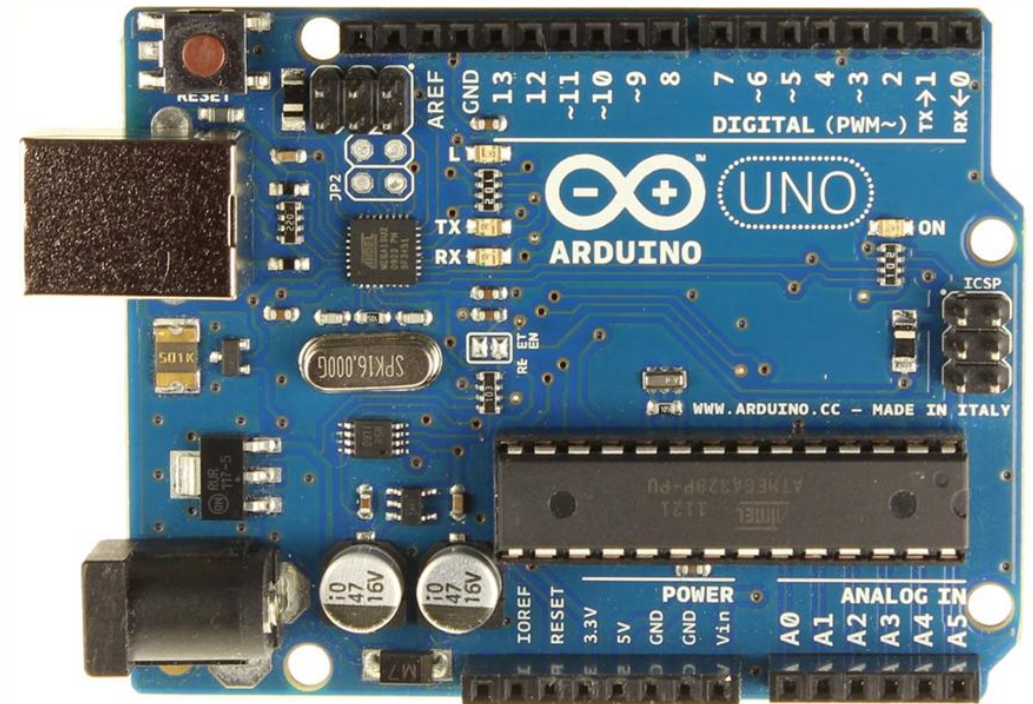
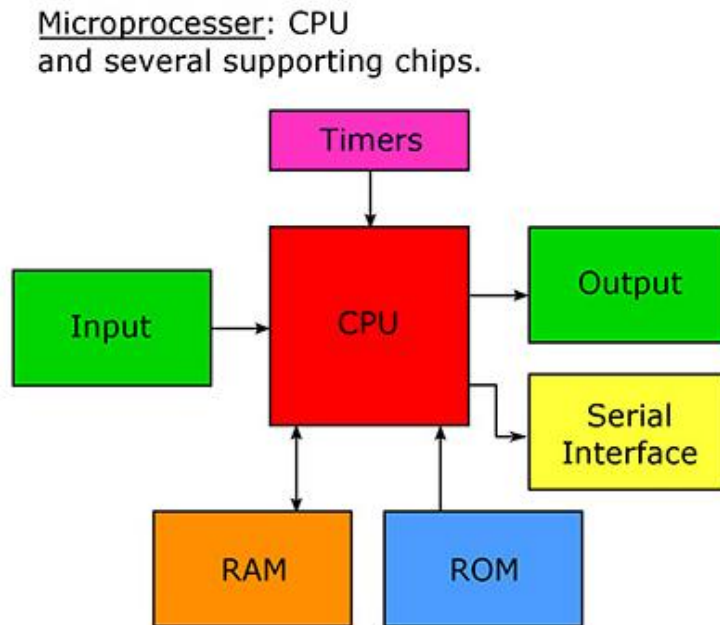


FIG 1. Arduino Uno SMD R3

OVERVIEW OF MICROCONTROLLERS: PROCESSOR, MEMORIES, AND INTERFACES

Microprocessor?

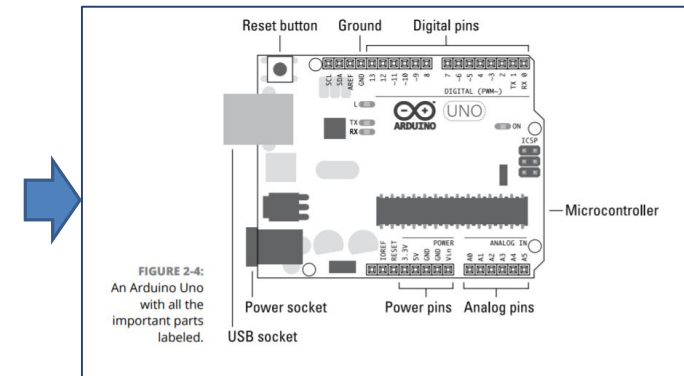
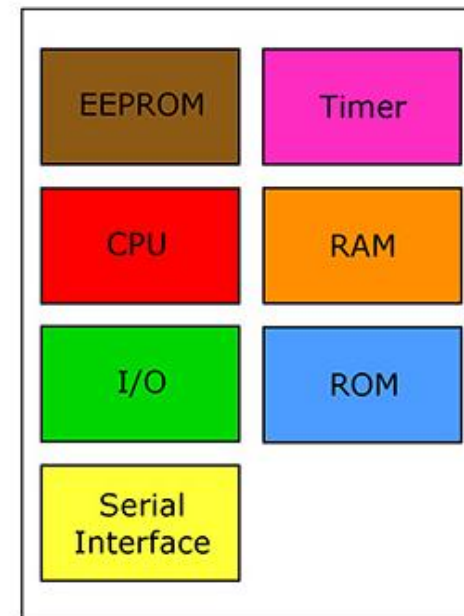
- Is a computer processor which incorporates the functions of a computer's central processing unit (CPU) on a single integrated circuit (IC).



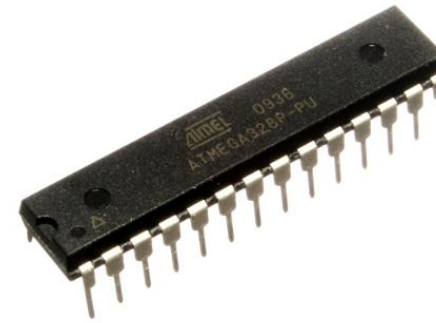
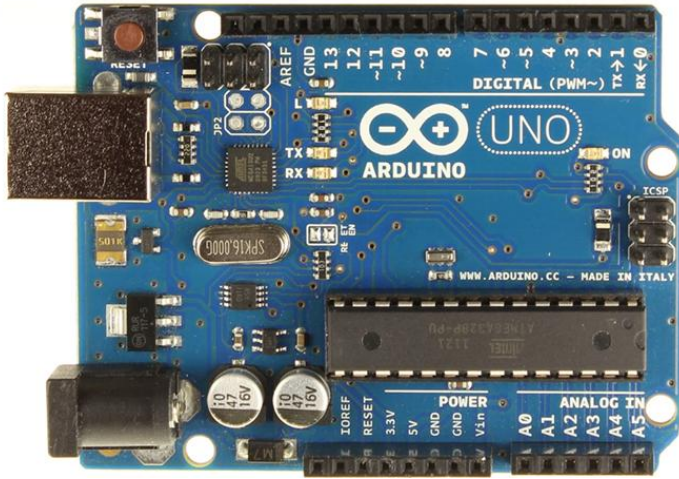
Microcontroller?

- Is a small computer on a single integrated circuit.
- A microcontroller contains one or more CPUs (processor cores) memory and programmable input/output peripherals.

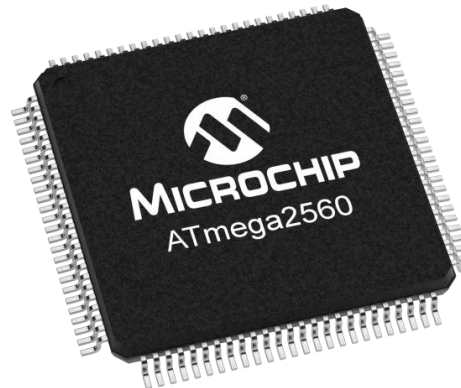
Microcontroller: CPU on a single chip.



AVR ARCHITECTURE (ATMEGA328, ATMEGA2560)



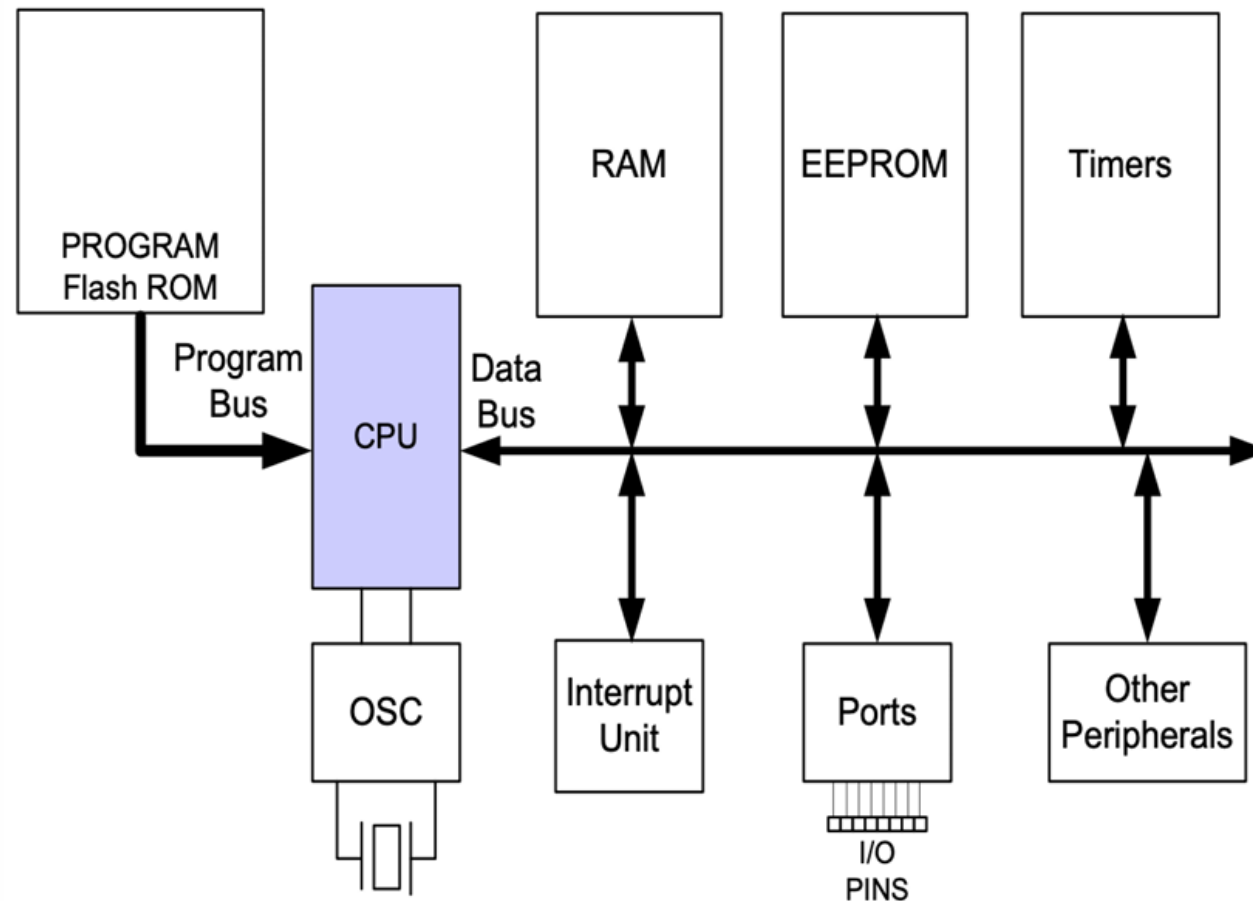
ATmega328



ATmega2560

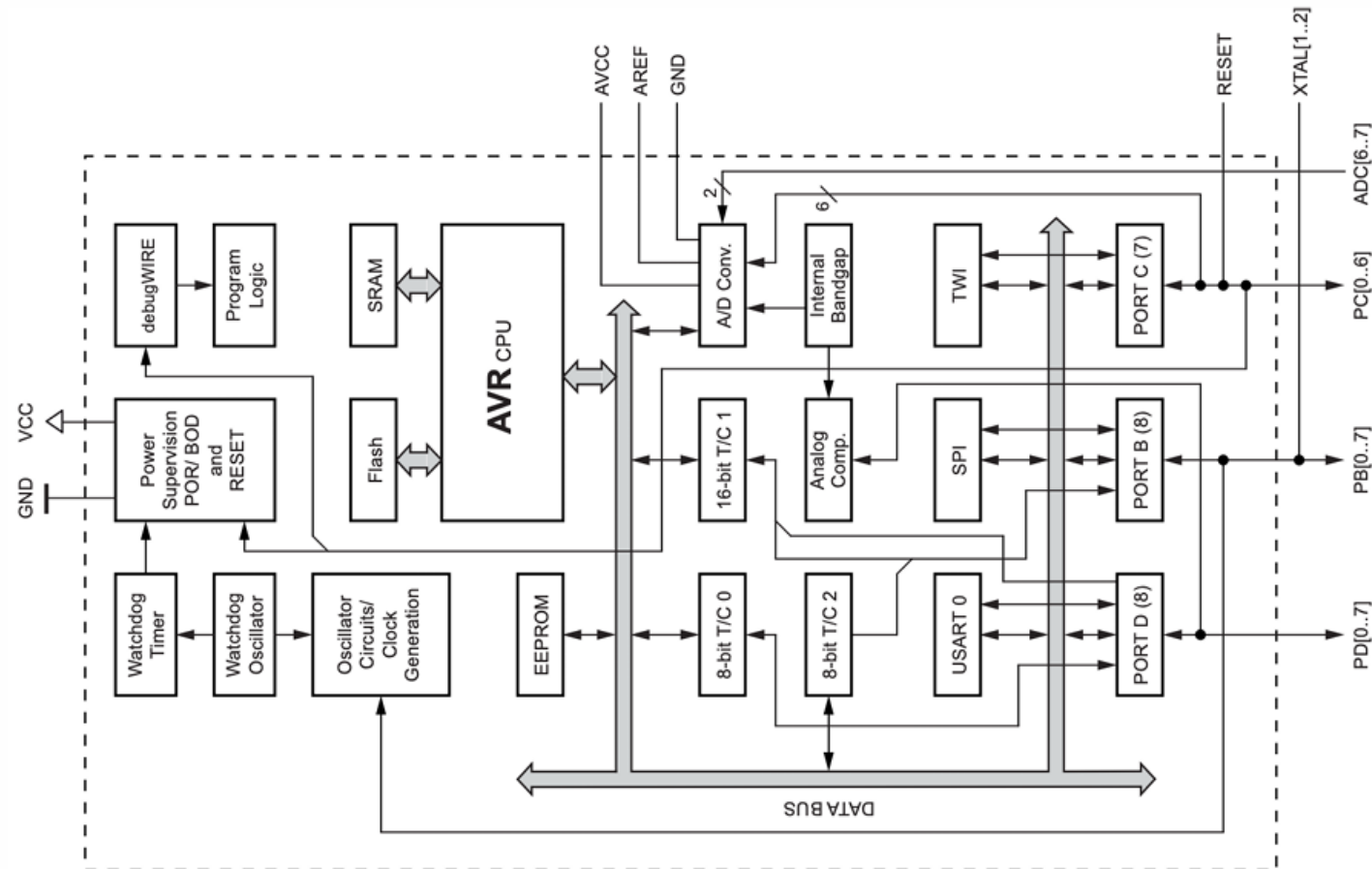
AVR ARCHITECTURE (ATMEGA328, ATMEGA2560)

Simplified view of an AVR microcontroller



AVR ARCHITECTURE (ATMEGA328, ATMEGA2560)

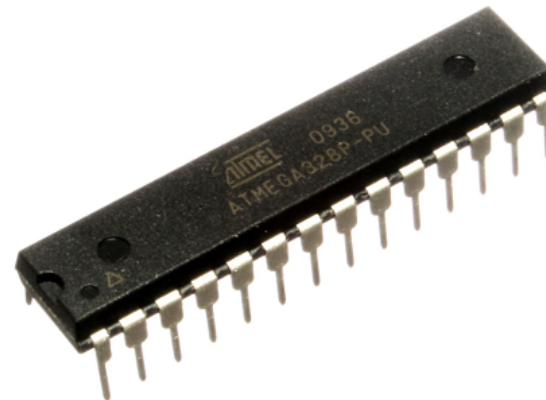
Atmega 328 architecture



AVR ARCHITECTURE (ATMEGA328, ATMEGA2560)

Atmega 328 architecture

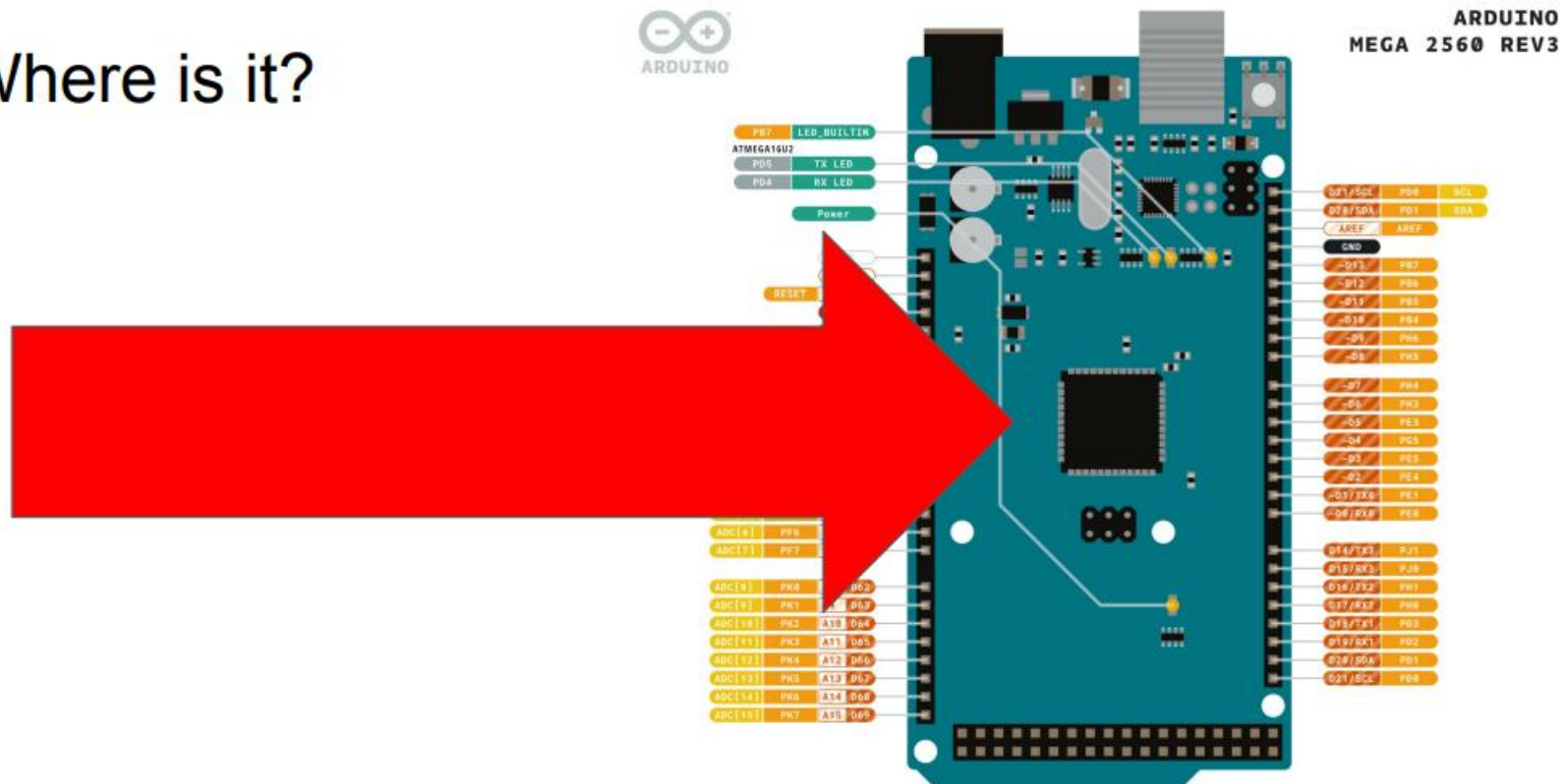
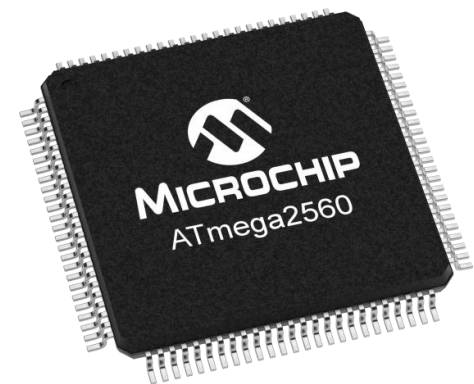
- Harvard architecture
- Programmable memory: 32K flash
- Data memory 2K SRAM
- Long term memory EEPROM: 1K
- Maximum CPU speed: 20 MHz
- 32 8-bit general purpose registers in SRAM memory space
- 3 8-bit Ports (B, C, D)—Each port controlled by 3 8-bit registers
- Digital Communication Peripherals:
1-UART, 2-SPI, 1-I2C
- Timers: 2 x 8-bit, 1 x 16-bit



AVR ARCHITECTURE (ATMEGA328, ATMEGA2560)

ATmega2560

What/Where is it?



AVR ARCHITECTURE (ATMEGA328, ATMEGA2560)

ATmega2560

Specs

Flash	EEPROM	RAM	General purpose I/O pins	16 Bit PWM Channels	Serial USARTS	ADC Channels
256KB	4KB	8KB	86	12	4	16

AVR ARCHITECTURE (ATMEGA328, ATMEGA2560)

ATmega2560

	ATmega328P	ATmega2560
Flash Memory	32 KB	256 KB
EEPROM	1 KB	4 KB
Programmable I/O Lines	23	86
Internal SRAM	2 KB	8 KB
Speed	16 MIPS at 16MHz	16 MIPS at 16MHz